**TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC**

**DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES**

**PROGRAMACIÓN DE BASE DE DATOS**

**TAREA 3:** TECNOLOGÍAS PARA CONECTIVIDAD A BASES DE DATOS: JDBC

**PROFESORA:**

ALEJANDRA GONZALEZ SÁNCHEZ

**NOMBRE:**

MONTERRUBIO LÓPEZ LUIS GAEL

**GRUPO:** 5701

*FECHA: 30-10-26*

# 1. Introducción y objetivo

Este documento explica cómo configurar y utilizar JDBC para conectar una aplicación Java con una base de datos PostgreSQL desplegada en Docker, y demuestra operaciones CRUD sobre registros de sensores IoT.

El contexto es la herramienta interna de NexoDev Solutions para el monitoreo de sensores, similar al proyecto Smart Factory Monitor. La aplicación registra lecturas de sensores (temperatura, humedad y presión) y permite consultarlas, actualizarlas y eliminarlas.

**Repositorio del proyecto:** https://github.com/luisgael-ml/smart-factory-monitor

**Competencia específica:** conectar una base de datos utilizando un lenguaje de programación actualizado (clave de actividad BDM-1301T01CE01A05).

**Objetivos específicos:**

1. Investigar y documentar la arquitectura, componentes y funcionamiento de JDBC.
2. Desplegar PostgreSQL con Docker y crear las tablas del proyecto.
3. Implementar una aplicación Java que realice operaciones CRUD mediante JDBC.
4. Ejecutar pruebas funcionales y registrar evidencias.
5. Gestionar el control de versiones con Git y GitHub y compartir el proyecto en un repositorio público.

# 2. Marco teórico: JDBC

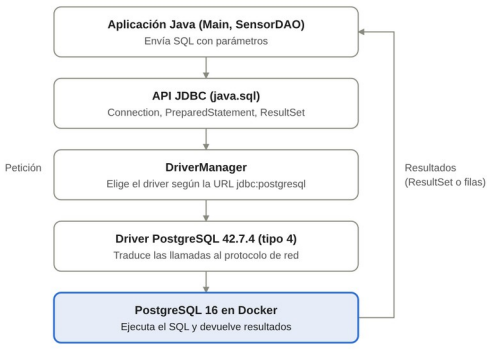
## 2.1 ¿Qué es JDBC?

JDBC (Java Database Connectivity) es la API estándar de Java para conectarse a bases de datos relacionales. Se incluye en el JDK dentro de los paquetes `java.sql` y `javax.sql`.

Su ventaja principal es la independencia del gestor: el mismo código Java funciona con PostgreSQL, MySQL, Oracle o SQL Server cambiando solo el driver y la URL de conexión.

## 2.2 Arquitectura

JDBC se organiza en capas: la aplicación Java llama a la **API JDBC**; el **DriverManager** elige el driver adecuado; el **driver** del fabricante traduce las llamadas al protocolo del gestor; y el **gestor de base de datos** ejecuta el SQL.




*Arquitectura JDBC: cada petición baja por las capas y el resultado regresa como ResultSet o filas afectadas.*

Existen dos modelos de despliegue:

| MODELO | CÓMO FUNCIONA | USO TÍPICO |
|---|---|---|
| Dos capas | La aplicación Java se conecta directamente al servidor de BD mediante el driver | Aplicaciones de escritorio y este proyecto |
| Tres capas | El cliente habla con un servidor intermedio (aplicación o API), y este se conecta a la BD vía JDBC | Aplicaciones web y empresariales |

### 2.3 Tipos de drivers JDBC

| TIPO | NOMBRE | DESCRIPCIÓN |
|---|---|---|
| 1 | Puente JDBC-ODBC | Traduce a ODBC. Obsoleto, eliminado desde Java 8 |
| 2 | API nativa | Usa librerías nativas del cliente de la BD. Depende de la plataforma |
| 3 | Protocolo de red | Pasa por un middleware que traduce al protocolo de la BD |
| 4 | Java puro (thin) | Escrito 100% en Java, habla directo el protocolo de la BD. Es el más usado |

El driver de PostgreSQL (`org.postgresql:postgresql`) es de **tipo 4**.

### 2.4 Componentes principales

| COMPONENTE | PAQUETE | FUNCIÓN |
|---|---|---|
| `DriverManager` | java.sql | Administra los drivers y crea conexiones con getConnection(url, usuario, contraseña) |
| `Connection` | java.sql | Representa la sesión abierta con la BD; controla transacciones (commit, rollback) |
| `Statement` | java.sql | Ejecuta SQL estático, sin parámetros |
| `PreparedStatement` | java.sql | Ejecuta SQL precompilado con parámetros ?; previene inyección SQL |
| `CallableStatement` | java.sql | Ejecuta procedimientos almacenados |
| `ResultSet` | java.sql | Contiene las filas devueltas por un SELECT; se recorre con next() |
| `SQLException` | java.sql | Excepción que reporta errores de acceso a datos |
| `DataSource` | javax.sql | Alternativa a DriverManager; permite pool de conexiones |

### 2.5 Funcionamiento: los 6 pasos de JDBC

1. **Cargar el driver:** desde JDBC 4.0 se registra solo al estar en el classpath.
2. **Abrir la conexión:** `DriverManager.getConnection(url, user, pass)`.
3. **Crear la sentencia:** `connection.prepareStatement(sql)`.
4. **Ejecutar:** `executeQuery()` para SELECT o `executeUpdate()` para INSERT, UPDATE y DELETE.
5. **Procesar resultados:** recorrer el `ResultSet`.
6. **Cerrar recursos:** con `try-with-resources` se cierran automáticamente.

Ejemplo mínimo:

```
String url = "jdbc:postgresql://localhost:5432/smartfactory";
try (Connection con = DriverManager.getConnection(url, "postgres", "admin123");
     PreparedStatement ps = con.prepareStatement("SELECT nombre, valor FROM sensores WHERE tipo = ?")) {
```

```
    ps.setString(1, "Temperatura");
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getString("nombre") + ": " + rs.getDouble("valor"));
        }
    }
} catch (SQLException e) {
    System.err.println("Error: " + e.getMessage());
}
```

### 2.6 Estructura de la URL de conexión

```
jdbc:postgresql://localhost:5432/smartfactory
```

| PARTE | SIGNIFICADO |
|---|---|
| `jdbc` | Protocolo |
| `postgresql` | Subprotocolo (identifica el driver) |
| `localhost` | Servidor donde corre la BD |
| `5432` | Puerto por defecto de PostgreSQL |
| `smartfactory` | Nombre de la base de datos |

## 3. Herramientas y requisitos

| HERRAMIENTA | VERSIÓN | USO EN EL PROYECTO |
|---|---|---|
| Docker Desktop | Actual | Desplegar PostgreSQL y ejecutar la aplicación en contenedores |
| PostgreSQL (imagen Docker) | 16 | Gestor de base de datos |
| JDK (imagen maven:3.9-eclipse-temurin-17) | 17 | Compilar y ejecutar la aplicación |
| Apache Maven | 3.9 | Gestionar dependencias (driver JDBC) y construir el proyecto |
| Driver JDBC PostgreSQL | 42.7.4 | Conexión entre Java y PostgreSQL (tipo 4) |
| Git + GitHub | Actual | Control de versiones y trabajo en equipo |

JDK y Maven pueden instalarse en la computadora, o usarse desde la imagen de Docker incluida en el proyecto, sin instalar nada adicional.

## 4. Configuración del entorno

### Paso 1. Desplegar PostgreSQL con Docker

El archivo `docker-compose.yml` define dos servicios: `postgres` (la base de datos) y `app` (la aplicación Java con JDK 17 y Maven).

```
services:
  # Base de datos PostgreSQL 16
  postgres:
    image: postgres:16
    container_name: pg-sensores
    environment:
      POSTGRES_PASSWORD: admin123
      POSTGRES_DB: smartfactory
    ports:
```

```
      - "5432:5432"
    volumes:
      # Crea la tabla y los datos iniciales la primera vez que arranca
      - ./sql/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d smartfactory"]
      interval: 3s
      retries: 10

  # Aplicación Java (JDK 17 + Maven dentro del contenedor, no requiere instalarlos)
  app:
    image: maven:3.9-eclipse-temurin-17
    working_dir: /app
    environment:
      DB_HOST: postgres
    volumes:
      - ./:/app
      - m2:/root/.m2          # guarda el driver descargado para no bajarlo cada vez
    command: mvn -q compile exec:java
    stdin_open: true
    tty: true
    depends_on:
      postgres:
        condition: service_healthy

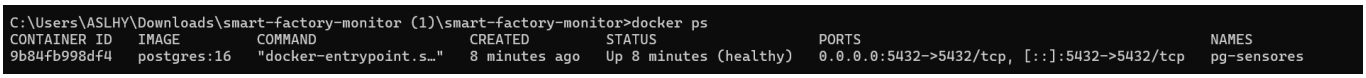
volumes:
  m2:
```

| PARÁMETRO | SIGNIFICADO |
|---|---|
| `container_name: pg-sensores` | Nombre del contenedor de la base de datos |
| `POSTGRES_PASSWORD: admin123` | Contraseña del usuario postgres |
| `POSTGRES_DB: smartfactory` | Crea la base de datos al iniciar |
| `ports: "5432:5432"` | Expone el puerto del contenedor en la máquina local |
| `docker-entrypoint-initdb.d` | Ejecuta init.sql la primera vez para crear la tabla |
| `healthcheck` | Indica cuándo PostgreSQL está listo para recibir conexiones |
| `DB_HOST: postgres` | Indica a la aplicación el servidor de la BD dentro de Docker |

Para levantar la base de datos y abrir la aplicación se ejecuta:

```
docker compose run --rm app
```

**Evidencia 1:** `docker ps` muestra el contenedor `pg-sensores` (postgres:16) en estado *Up (healthy)* con el puerto 5432 expuesto.




*Contenedor de PostgreSQL en ejecución.*

### Paso 2. Crear la tabla de sensores

El script `sql/init.sql` crea la tabla e inserta tres sensores de ejemplo. Docker lo ejecuta automáticamente al crear el contenedor.

```
-- Tabla de sensores del proyecto Smart Factory Monitor
CREATE TABLE IF NOT EXISTS sensores (
    id        SERIAL PRIMARY KEY,           -- identificador autoincremental
    nombre    VARCHAR(50)  NOT NULL,        -- nombre del sensor
```

```
    tipo      VARCHAR(30)  NOT NULL,        -- temperatura, humedad, presión...
    ubicacion VARCHAR(50),                  -- área de la planta
    valor     NUMERIC(10,2),                -- última lectura
    fecha     TIMESTAMP DEFAULT NOW()       -- fecha de registro
);

-- Datos iniciales
INSERT INTO sensores (nombre, tipo, ubicacion, valor) VALUES
('SEN-T01', 'Temperatura', 'Línea 1', 24.50),
('SEN-H01', 'Humedad', 'Almacén', 55.00),
('SEN-P01', 'Presión', 'Caldera', 101.30);
```

**Evidencia 2:** la tabla `sensores` creada con sus 3 registros iniciales, consultada desde la aplicación.

```
Opción: 2
ID   NOMBRE     TIPO         UBICACIÓN       VALOR  FECHA
1    SEN-T01    Temperatura  Línea 1          24.50  2026-10-30 05:49:09.163544
2    SEN-H01    Humedad      Almacén          55.00  2026-10-30 05:49:09.163544
3    SEN-P01    Presión      Caldera         101.30  2026-10-30 05:49:09.163544
```

*Tabla sensores con sus registros iniciales.*

### Paso 3. Configurar el proyecto Maven y el driver JDBC

El archivo `pom.xml` declara Java 17, el driver de PostgreSQL y el plugin que ejecuta la clase principal. Maven descarga el driver automáticamente al compilar.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.nexodev</groupId>
    <artifactId>smart-factory-monitor</artifactId>
    <version>1.0</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <!-- Driver JDBC de PostgreSQL (tipo 4) -->
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <version>42.7.4</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- Permite ejecutar con: mvn compile exec:java -->
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.5.0</version>
                <configuration>
```

```
                    <mainClass>com.nexodev.Main</mainClass>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

# 5. Implementación

La aplicación sigue el patrón **DAO (Data Access Object)**: separa la conexión, el modelo de datos y las operaciones SQL en clases distintas, lo que hace el código ordenado y fácil de mantener.

### 5.1 Estructura del proyecto

```
smart-factory-monitor/
├── pom.xml
├── docker-compose.yml
├── sql/init.sql
└── src/main/java/com/nexodev/
    ├── Conexion.java    → abre la conexión JDBC
    ├── Sensor.java      → modelo (representa una fila de la tabla)
    ├── SensorDAO.java   → operaciones CRUD
    └── Main.java        → menú de consola para probar
```

### 5.2 Conexion.java

Centraliza los datos de conexión y devuelve un objeto `Connection`. El servidor se toma de la variable `DB_HOST`: `localhost` al ejecutar en la computadora y `postgres` dentro de Docker.

```
package com.nexodev;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * Clase responsable de crear la conexión con PostgreSQL mediante JDBC.
 */
public class Conexion {

    // Servidor de la BD: "localhost" al correr en la PC, o el nombre del
    // servicio "postgres" cuando la app corre dentro de Docker (variable DB_HOST)
    private static final String HOST = System.getenv().getOrDefault("DB_HOST", "localhost");

    // URL JDBC: protocolo:subprotocolo://servidor:puerto/base_de_datos
    private static final String URL = "jdbc:postgresql://" + HOST + ":5432/smartfactory";
    private static final String USUARIO = "postgres";
    private static final String PASSWORD = "admin123";

    /**
     * Abre y devuelve una conexión nueva.
     * El driver se registra solo (JDBC 4.0+) por estar en el classpath vía Maven.
     */
    public static Connection getConexion() throws SQLException {
        return DriverManager.getConnection(URL, USUARIO, PASSWORD);
    }
}
```

### 5.3 Sensor.java

Representa un registro de la tabla `sensores`.

```
package com.nexodev;

import java.sql.Timestamp;

/**
 * Modelo de datos: cada objeto equivale a una fila de la tabla sensores.
 */
public class Sensor {
    private int id;
    private String nombre;
    private String tipo;
    private String ubicacion;
    private double valor;
    private Timestamp fecha;

    public Sensor() { }

    public Sensor(String nombre, String tipo, String ubicacion, double valor) {
        this.nombre = nombre;
        this.tipo = tipo;
        this.ubicacion = ubicacion;
        this.valor = valor;
    }

    // Getters y setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getTipo() { return tipo; }
    public void setTipo(String tipo) { this.tipo = tipo; }
    public String getUbicacion() { return ubicacion; }
    public void setUbicacion(String ubicacion) { this.ubicacion = ubicacion; }
    public double getValor() { return valor; }
    public void setValor(double valor) { this.valor = valor; }
    public Timestamp getFecha() { return fecha; }
    public void setFecha(Timestamp fecha) { this.fecha = fecha; }

    @Override
    public String toString() {
        return String.format("%-4d %-10s %-12s %-12s %8.2f  %s",
                id, nombre, tipo, ubicacion, valor, fecha);
    }
}
```

### 5.4 SensorDAO.java

Contiene las cuatro operaciones CRUD. Todas usan `PreparedStatement` (evita inyección SQL) y `try-with-resources` (cierra la conexión automáticamente).

```
package com.nexodev;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

```
/**
 * Data Access Object: concentra todas las operaciones SQL sobre la tabla sensores.
 */
public class SensorDAO {

    /** CREATE: inserta un sensor nuevo. Devuelve true si se insertó. */
    public boolean insertar(Sensor s) {
        String sql = "INSERT INTO sensores (nombre, tipo, ubicacion, valor) VALUES (?, ?, ?, ?)";
        try (Connection con = Conexion.getConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, s.getNombre());     // reemplaza el primer ?
            ps.setString(2, s.getTipo());
            ps.setString(3, s.getUbicacion());
            ps.setDouble(4, s.getValor());
            return ps.executeUpdate() > 0;      // filas afectadas
        } catch (SQLException e) {
            System.err.println("Error al insertar: " + e.getMessage());
            return false;
        }
    }

    /** READ: devuelve todos los sensores ordenados por id. */
    public List<Sensor> listar() {
        List<Sensor> lista = new ArrayList<>();
        String sql = "SELECT id, nombre, tipo, ubicacion, valor, fecha FROM sensores ORDER BY id";
        try (Connection con = Conexion.getConexion();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {                 // recorre fila por fila
                Sensor s = new Sensor();
                s.setId(rs.getInt("id"));
                s.setNombre(rs.getString("nombre"));
                s.setTipo(rs.getString("tipo"));
                s.setUbicacion(rs.getString("ubicacion"));
                s.setValor(rs.getDouble("valor"));
                s.setFecha(rs.getTimestamp("fecha"));
                lista.add(s);
            }
        } catch (SQLException e) {
            System.err.println("Error al listar: " + e.getMessage());
        }
        return lista;
    }

    /** UPDATE: actualiza el valor de lectura de un sensor por id. */
    public boolean actualizarValor(int id, double nuevoValor) {
        String sql = "UPDATE sensores SET valor = ?, fecha = NOW() WHERE id = ?";
        try (Connection con = Conexion.getConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setDouble(1, nuevoValor);
            ps.setInt(2, id);
            return ps.executeUpdate() > 0;
        } catch (SQLException e) {
            System.err.println("Error al actualizar: " + e.getMessage());
            return false;
        }
    }

    /** DELETE: elimina un sensor por id. */
    public boolean eliminar(int id) {
```

```
        String sql = "DELETE FROM sensores WHERE id = ?";
        try (Connection con = Conexion.getConexion();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            return ps.executeUpdate() > 0;
        } catch (SQLException e) {
            System.err.println("Error al eliminar: " + e.getMessage());
            return false;
        }
    }
}
```

### 5.5 Main.java

Menú de consola que permite probar cada operación.

```
package com.nexodev;

import java.util.Scanner;

/**
 * Punto de entrada: menú para probar el CRUD de sensores.
 */
public class Main {
    public static void main(String[] args) {
        SensorDAO dao = new SensorDAO();
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println("\n=== SMART FACTORY MONITOR ===");
            System.out.println("1. Insertar sensor");
            System.out.println("2. Listar sensores");
            System.out.println("3. Actualizar lectura");
            System.out.println("4. Eliminar sensor");
            System.out.println("0. Salir");
            System.out.print("Opción: ");
            opcion = Integer.parseInt(sc.nextLine());

            switch (opcion) {
                case 1 -> {
                    System.out.print("Nombre: ");    String nombre = sc.nextLine();
                    System.out.print("Tipo: ");      String tipo = sc.nextLine();
                    System.out.print("Ubicación: "); String ubicacion = sc.nextLine();
                    System.out.print("Valor: ");     double valor = Double.parseDouble(sc.nextLine());
                    boolean ok = dao.insertar(new Sensor(nombre, tipo, ubicacion, valor));
                    System.out.println(ok ? "Sensor insertado." : "No se pudo insertar.");
                }
                case 2 -> {
                    System.out.printf("%-4s %-10s %-12s %-12s %8s  %s%n",
                            "ID", "NOMBRE", "TIPO", "UBICACIÓN", "VALOR", "FECHA");
                    dao.listar().forEach(System.out::println);
                }
                case 3 -> {
                    System.out.print("ID del sensor: "); int id = Integer.parseInt(sc.nextLine());
                    System.out.print("Nuevo valor: ");   double v = Double.parseDouble(sc.nextLine());
                    System.out.println(dao.actualizarValor(id, v) ? "Lectura actualizada." : "ID no
encontrado.");
                }
                case 4 -> {
```

```
                    System.out.print("ID del sensor: "); int id = Integer.parseInt(sc.nextLine());
                    System.out.println(dao.eliminar(id) ? "Sensor eliminado." : "ID no encontrado.");
                }
                case 0 -> System.out.println("Hasta luego.");
                default -> System.out.println("Opción inválida.");
            }
        } while (opcion != 0);
        sc.close();
    }
}
```

### 5.6 Ejecución

Con Docker (sin instalar Java ni Maven):

```
docker compose run --rm app
```

Con JDK 17 y Maven instalados en la computadora:

```
docker compose up -d postgres
mvn compile exec:java
```

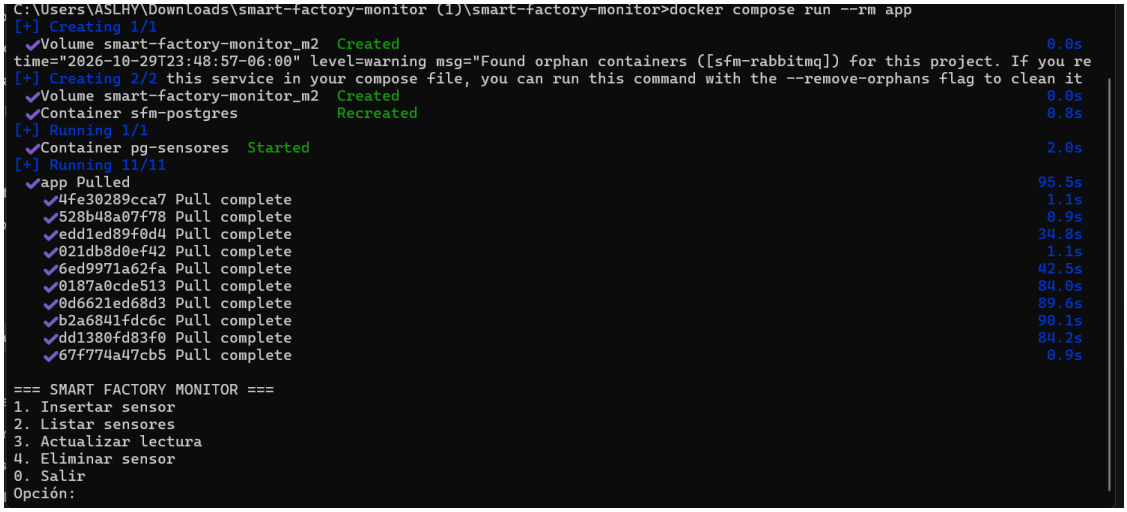
## 6. Pruebas funcionales

Cada operación se probó desde el menú de la aplicación. Los resultados se verificaron con la opción de listar.

| NO. | OPERACIÓN | ENTRADA | RESULTADO ESPERADO | RESULTADO OBTENIDO |
|---|---|---|---|---|
| P1 | Conexión | Ejecutar la aplicación | Se abre la conexión sin SQLException | Correcto: conexión establecida, menú abierto sin errores |
| P2 | Listar (READ) | Opción 2 | Se muestran los 3 sensores iniciales | Correcto: se listaron SEN-T01, SEN-H01 y SEN-P01 |
| P3 | Insertar (CREATE) | SEN-T02, Temperatura, Línea 2, 26.80 | Mensaje "Sensor insertado" y aparece con id 4 | Correcto: mensaje "Sensor insertado." |
| P4 | Actualizar (UPDATE) | id 1, valor 30.25 | Mensaje "Lectura actualizada" y el valor cambia | Correcto: mensaje "Lectura actualizada." |
| P5 | Eliminar (DELETE) | id 3 | Mensaje "Sensor eliminado" y desaparece de la tabla | Correcto: mensaje "Sensor eliminado." |
| P6 | Id inexistente | Actualizar id 99 | Mensaje "ID no encontrado" | Correcto: mensaje "ID no encontrado."; la tabla final muestra los ids 1 (30.25), 2 y 4 |
| P7 | Error de conexión | Detener PostgreSQL y listar | Mensaje de error controlado, sin cierre abrupto | Correcto: "Error al listar: The connection attempt failed." y el menú sigue funcionando |

### Evidencias

**Evidencia 3 — P1:** `docker compose run --rm app` inicia PostgreSQL, descarga la imagen con JDK 17 y Maven, y la aplicación abre el menú tras conectarse por JDBC.




*Conexión establecida y menú de la aplicación.*

**Evidencia 4 — P3 (CREATE):** inserción del sensor SEN-T02.

```
Opción: 1
Nombre: SEN-T02
Tipo: Temperatura
Ubicación: Línea 2
Valor: 26.80
Sensor insertado.
```

*Inserción de un sensor nuevo.*

**Evidencia 5 — P4 (UPDATE):** actualización de la lectura del sensor 1 a 30.25.

```
Opción: 3
ID del sensor: 1
Nuevo valor: 30.25
Lectura actualizada.
```

*Actualización de una lectura.*

**Evidencia 6 — P5 (DELETE):** eliminación del sensor 3.

```
Opción: 4
ID del sensor: 3
Sensor eliminado.
```

*Eliminación de un sensor.*

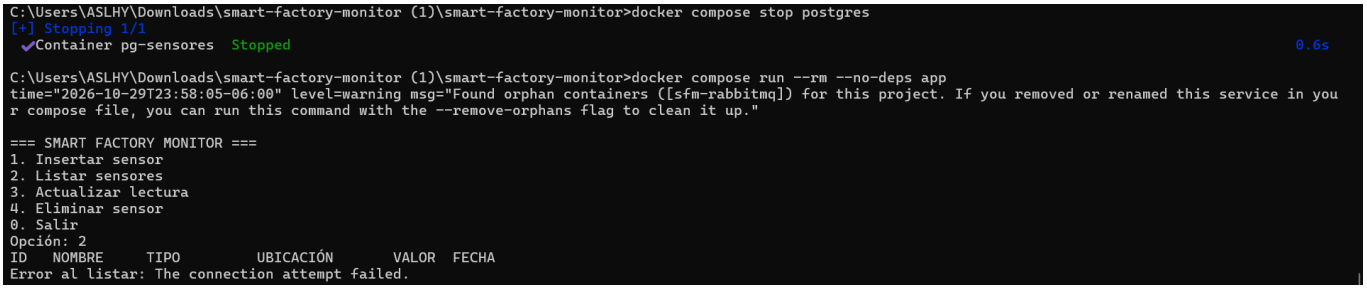
**Evidencia 7 — P6:** el id 99 no existe y la aplicación lo informa. El listado final confirma las tres operaciones: id 1 con 30.25, id 4 insertado e id 3 eliminado.

```
Opción: 3
ID del sensor: 99
Nuevo valor: 10
ID no encontrado.

=== SMART FACTORY MONITOR ===
1. Insertar sensor
2. Listar sensores
3. Actualizar lectura
4. Eliminar sensor
0. Salir
Opción: 2
ID   NOMBRE     TIPO         UBICACIÓN       VALOR  FECHA
1    SEN-T01    Temperatura  Línea 1         30.25  2026-10-30 05:54:52.13859
2    SEN-H01    Humedad      Almacén         55.00  2026-10-30 05:49:09.163544
4    SEN-T02    Temperatura  Línea 2         26.80  2026-10-30 05:54:27.303537
```

*Id inexistente y listado final de sensores.*

**Evidencia 7 — P7:** con PostgreSQL detenido, la aplicación muestra un error controlado y el menú sigue funcionando.




*Manejo controlado del error de conexión.*

# 7. Control de versiones y recursos compartidos

La actividad se desarrolló de forma individual. Git y GitHub se usaron para registrar el historial de cambios del proyecto y publicarlo en un repositorio público.

**Repositorio:** https://github.com/luisgael-ml/smart-factory-monitor

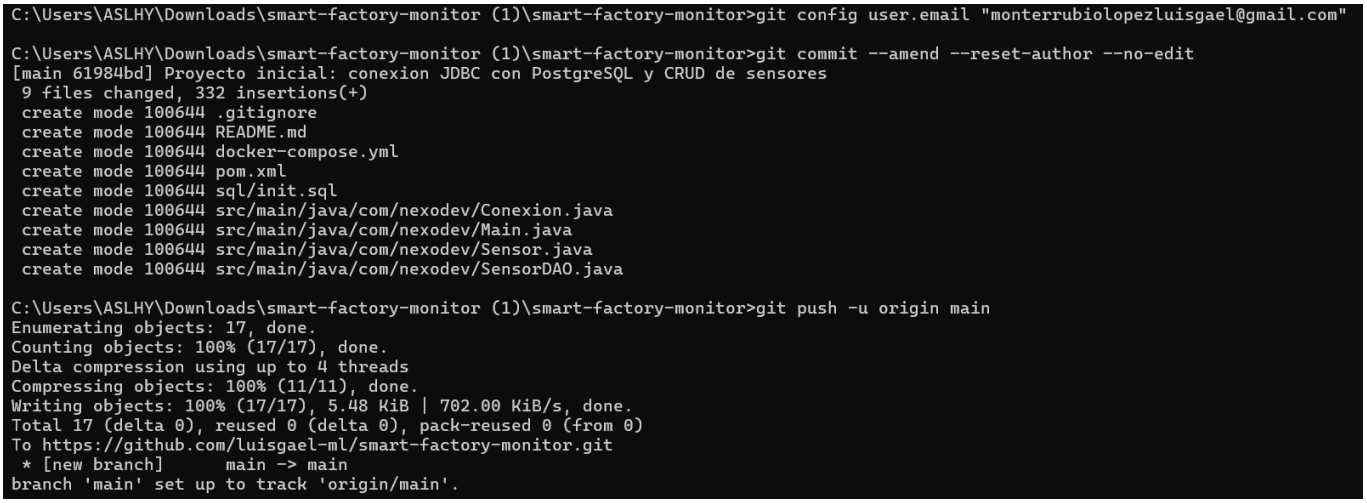
## 7.1 Flujo de trabajo con Git

```
git init
git add .
git commit -m "Proyecto inicial: conexion JDBC con PostgreSQL y CRUD de sensores"
git branch -M main
git remote add origin https://github.com/luisgael-ml/smart-factory-monitor.git
git push -u origin main
```

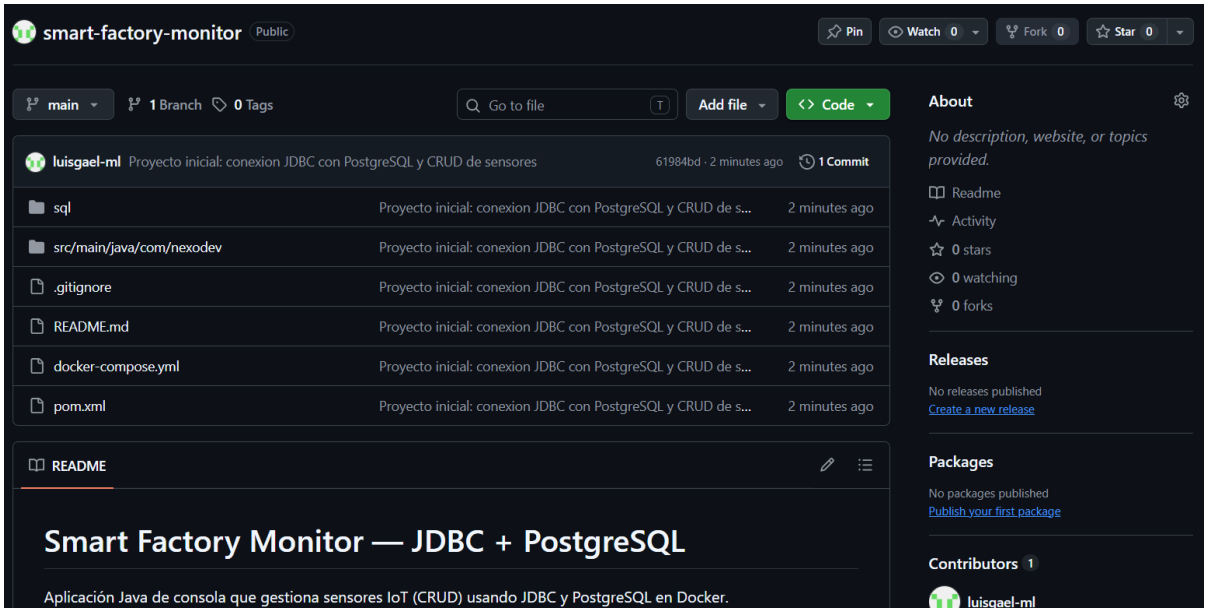
## 7.2 Recursos compartidos

El repositorio incluye el código fuente, el script SQL, el archivo `docker-compose.yml` y un `README.md` con las instrucciones de ejecución. Cualquier compañero puede reproducir el proyecto con un solo comando (`docker compose run --rm app`), sin instalar Java ni Maven.

**Evidencia 8:** el commit se registró a nombre de Luis Gael Monterrubio López y se publicó en la rama `main` del repositorio en GitHub.




*Commit y publicación del proyecto en GitHub.*




*Repositorio público smart-factory-monitor en GitHub con el código del proyecto.*

### 7.3 Reflexión sobre la colaboración

Aunque desarrollé esta actividad de forma individual, la preparé para que otros puedan aprovecharla. Dejé el repositorio público, documenté cada paso y configuré Docker para que el proyecto funcione igual en cualquier computadora. Así, un compañero que tenga problemas con su instalación de Java o PostgreSQL puede revisar una solución completa y funcional.

Considero que compartir código reproducible es una forma concreta de colaborar: ahorra tiempo a otros y permite comparar soluciones. En un trabajo en equipo, este mismo flujo con Git se ampliaría con ramas por integrante y *pull requests* revisados por otro compañero antes de integrar los cambios.

## 8. Conclusiones

JDBC permitió conectar una aplicación Java con PostgreSQL usando solo la API estándar del JDK y un driver tipo 4 agregado con Maven.

El patrón DAO separó la conexión, el modelo y las consultas, lo que facilitó probar cada operación por separado. El uso de `PreparedStatement` protegió la aplicación contra inyección SQL, y `try-with-resources` garantizó el cierre de conexiones.

Docker simplificó el despliegue: un solo comando levantó PostgreSQL y ejecutó la aplicación con JDK 17 y Maven, sin instalarlos en la computadora.

## Referencias

Oracle. (s.f.). *JDBC Basics — The Java Tutorials*. https://docs.oracle.com/javase/tutorial/jdbc/basics/

Oracle. (s.f.). *Package java.sql — Java SE 17 API*. https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/package-summary.html

PostgreSQL Global Development Group. (s.f.). *PostgreSQL JDBC Driver Documentation*. https://jdbc.postgresql.org/documentation/

Docker Inc. (s.f.). *postgres — Docker Official Image*. https://hub.docker.com/_/postgres